Analyse FFT et traitement du signal dans `pc_analyzer`

Projet Radar Doppler 24.125 GHz

Mars 2026

Table des matières

1 Objectif de ce document **3**

2 Vue système : de l'onde radar à la vitesse affichée **3**
2.1 Chaîne fonctionnelle . . . 3
2.2 Relation Doppler utilisée . . . 3

3 Paramètres fondamentaux de `pc_analyzer` **3**

4 Transport UDP et reconstruction du signal **3**
4.1 Fragmentation côté réseau . . . 3
4.2 Fenêtre glissante locale . . . 4

5 Pré-traitement temporel avant FFT **4**
5.1 Notations et cadre mathématique . . . 4
5.2 Suppression de la composante continue (DC removal) . . . 4
5.3 Fenêtre de Hann : forme, rôle, et compromis . . . 4
5.4 Zero-padding : ce que c'est, et ce que ce n'est pas . . . 5
5.5 Exemple numérique (section 5) . . . 5

6 FFT, magnitude et échelle en dB **5**
6.1 Transformée discrète en fréquence . . . 5
6.2 Correspondance bin $\leftrightarrow$ fréquence . . . 6
6.3 Module complexe et passage en dB . . . 6
6.4 Lissage exponentiel bin par bin . . . 6
6.5 Exemple numérique (section 6) . . . 6

7 Détection CFAR et robustification bruit **7**
7.1 CA-CFAR : détail de la construction du seuil . . . 7
7.2 Traitement des perturbations secteur et harmoniques . . . 7
7.3 Notch paramétrique . . . 7
7.4 Plancher de bruit adaptatif : modèle dynamique . . . 7
7.5 Exemple numérique (section 7) . . . 8

8 Extraction de la cible et estimation cinématique **8**
8.1 Du masque binaire au pic cible . . . 8
8.2 Conversion fréquence $\rightarrow$ vitesse . . . 8
8.3 SNR de la cible . . . 9
8.4 Filtre de Kalman : dérivation des équations utilisées . . . 9
8.5 Exemple numérique (section 8) . . . 9

9 Publication des résultats et rendu GUI **10**
9.1 Données partagées 10
9.2 Sorties visuelles 10

10 Lecture “professeur” : compromis et limites **10**
10.1 Résolution et latence 10
10.2 Ce que CFAR résout, et ce qu’il ne résout pas 10
10.3 Pourquoi vitesse brute + vitesse filtrée 11

11 Conclusion **11**

1 Objectif de ce document

Ce document explique, en profondeur, la chaîne de traitement du signal du programme `pc_analyzer` (`radar_analyzer.cpp`), depuis l'acquisition ADC côté STM32 jusqu'au résultat final affiché à l'opérateur (fréquence Doppler, vitesse, SNR, spectre, spectrogramme, tendance).

L'objectif est double :

— **Technique** : comprendre exactement ce que fait le code.

— **Théorique** : relier chaque choix d'implémentation aux principes DSP/radar.

2 Vue système : de l'onde radar à la vitesse affichée

2.1 Chaîne fonctionnelle

Cible en mouvement → Décalage Doppler f_d → Signal IF analogique → ADC STM32 (100 kHz) → Hops UDP –

2.2 Relation Doppler utilisée

Le code utilise le facteur :

$$f_d \approx 161 \cdot v \quad \text{(Hz pour } v \text{ en m/s)}$$

soit :

$$v = \frac{f_d}{161}$$

Ce facteur provient de la formule monostatique :

$$f_d = \frac{2vf_0}{c}$$

avec $f_0 = 24.125\,\text{GHz}$.

3 Paramètres fondamentaux de `pc_analyzer`

Le code fixe les constantes de base suivantes :

```
FS = 100000.0f
STM32_FRAME_SIZE = 4096
STM32_HOP_SIZE = 1024
UDP_PORT = 5555
DOPPLER_SCALE = 161.0f
```

Interprétation physique :

— Un hop de 1024 échantillons couvre 1024/100000 = 10.24 ms.

— Une fenêtre de 4096 couvre 40.96 ms.

— Le moteur FFT PC est configurable (1024 → 65536 points), avec valeur par défaut 8192.

4 Transport UDP et reconstruction du signal

4.1 Fragmentation côté réseau

Chaque hop ADC (1024 × uint16 = 2048 octets) est fragmenté. Côté PC, le thread UDP décode un en-tête de 12 octets :

— `seq` : numéro de séquence hop,

— `frag_id`, `frag_count`,
— `offset`, `payload_len`.

Le réassemblage utilise un bitmap de fragments pour :

— accepter les arrivées hors ordre,
— ignorer les doublons,
— détecter les hops incomplets/perdus.

4.2 Fenêtre glissante locale

Une fois un hop complet reçu :

1. on décale `rolling_adc` de 1024 échantillons (`memmove`),
2. on insère le nouveau hop en fin de fenêtre.

Cette opération crée une analyse **overlap 75%** :

$$\text{pas temporel} = 1024, \quad \text{taille fenêtre} = 4096$$

ce qui améliore la réactivité d'affichage sans sacrifier la stabilité fréquentielle.

5 Pré-traitement temporel avant FFT

5.1 Notations et cadre mathématique

Dans cette section, nous notons :

— $x[n]$: échantillon ADC brut (après reconstruction de fenêtre glissante),
— $N_w = 4096$: longueur physique de la fenêtre glissante traitée,
— N_{FFT} : taille de FFT demandée ($N_{FFT} \geq N_w$, puissance de 2),
— $F_s = 100\,000$ Hz : fréquence d'échantillonnage.

5.2 Suppression de la composante continue (DC removal)

Le code estime la moyenne temporelle puis centre le signal :

$$\mu = \frac{1}{N_w} \sum_{n=0}^{N_w-1} x[n], \qquad x_c[n] = x[n] - \mu.$$

Pourquoi c'est crucial. Sans cette étape, la composante quasi-statique de l'ADC (offset analogique + offset numérique) concentre une énergie importante autour du bin $k = 0$. Via fuite spectrale, cette énergie se propage dans les bins voisins et réduit fortement la sensibilité de détection aux faibles Doppler.

Lecture physique. On ne cherche pas l'énergie absolue du front-end, mais la *modulation Doppler*. Le centrage transforme donc le problème en "analyser des variations autour d'un point d'équilibre".

5.3 Fenêtre de Hann : forme, rôle, et compromis

Le fenêtrage est :

$$w[n] = \frac{1}{2}\left(1 - \cos\left(\frac{2\pi n}{N_w - 1}\right)\right), \quad 0 \leq n < N_w$$

et

$$x_w[n] = x_c[n]\, w[n].$$

Composants de la formule.

— Le terme $\cos(\cdot)$ impose une transition douce aux extrémités.

— Le facteur $1/2$ normalise la plage vers $[0, 1]$.

— La division par $N_w - 1$ aligne exactement les bords $n = 0$ et $n = N_w - 1$ à zéro.

Pourquoi fenêtrer en radar Doppler. La FFT suppose implicitement un signal périodique sur la fenêtre analysée. Sans fenêtre, la couture entre fin et début crée une discontinuité, ce qui étale l'énergie d'une raie dans de nombreux bins (*spectral leakage*). La Hann réduit drastiquement les lobes secondaires, au prix d'un lobe principal plus large.

Point d'implémentation important. Le code calcule $w[n]$ sur $N_w = 4096$, même si $N_{FFT} > 4096$. C'est correct : la fenêtre doit épouser la portion effectivement observée du signal, puis le padding intervient *après*.

5.4 Zero-padding : ce que c'est, et ce que ce n'est pas

Si $N_{FFT} > N_w$, le code applique :

$$x_p[n] = \begin{cases} x_w[n], & 0 \le n < N_w \\ 0, & N_w \le n < N_{FFT}. \end{cases}$$

Interprétation rigoureuse. Le zero-padding n'augmente pas l'information (pas de nouvelle mesure), mais augmente la densité d'échantillonnage de la transformée discrète en fréquence. En pratique, cela facilite la lecture du pic, l'affichage, et la stabilité des méthodes de détection basées sur le "maximum de bin".

5.5 Exemple numérique (section 5)

Prenons un bloc de 4096 échantillons ADC avec :

$$\mu = 2042.7 \text{ LSB}, \quad x[1000] = 2190 \text{ LSB}.$$

Après centrage :

$$x_c[1000] = 2190 - 2042.7 = 147.3 \text{ LSB}.$$

Supposons $w[1000] = 0.31$ (Hann). Alors :

$$x_w[1000] = 147.3 \times 0.31 \approx 45.7 \text{ LSB}.$$

Si $N_{FFT} = 8192$, les 4096 échantillons suivants sont zéro :

$$x_p[4096] = x_p[4097] = \cdots = x_p[8191] = 0.$$

On a donc la même information physique que sur 4096 points, mais un maillage spectral plus fin.

6 FFT, magnitude et échelle en dB

6.1 Transformée discrète en fréquence

Le moteur `kissfft::rfft` calcule :

$$X[k] = \sum_{n=0}^{N_{FFT}-1} x_p[n] e^{-j2\pi kn/N_{FFT}}, \quad 0 \le k < N_{FFT}.$$

Comme le signal est réel, le spectre est conjugué-symétrique, et l'on exploite la moitié positive (0 à $N_{FFT}/2$).

6.2 Correspondance bin $\leftrightarrow$ fréquence

Chaque bin k représente :

$$f_k = k\frac{F_s}{N_{FFT}}, \qquad \Delta f = \frac{F_s}{N_{FFT}}.$$

Exemple. Pour $F_s = 100\,\text{kHz}$, $N_{FFT} = 8192$:

$$\Delta f \approx 12.207\,\text{Hz/bin}.$$

6.3 Module complexe et passage en dB

Le code calcule :

$$|X[k]| = \sqrt{\Re(X[k])^2 + \Im(X[k])^2},$$

puis

$$S[k]_{\text{dB}} = 20\log_{10}\big(|X[k]| + \varepsilon\big), \quad \varepsilon = 10^{-12}.$$

Pourquoi $20\log_{10}$ et pas $10\log_{10}$. $S[k]$ est un *module d'amplitude*. Pour une grandeur de puissance, on utiliserait $10\log_{10}$. Ici on convertit une amplitude, d'où le facteur 20.

Rôle de ε. Il garantit la définition numérique du logarithme même si $|X[k]| \approx 0$, sans altérer la dynamique utile.

6.4 Lissage exponentiel bin par bin

Le spectre lissé suit :

$$S_s^{(m)}[k] = (1-\alpha)S_s^{(m-1)}[k] + \alpha S^{(m)}[k],$$

où m est l'indice temporel de hop traité.

Constante de temps effective. Avec un pas temporel $T_h = 10.24$ ms (hop de 1024), une approximation de la constante de temps est :

$$\tau \approx \frac{T_h}{\alpha}.$$

Donc α petit donne un spectre très stable (mais moins réactif).

6.5 Exemple numérique (section 6)

Considérons $F_s = 100000$ Hz.

Cas A : $N_{FFT} = 8192$.

$$\Delta f = \frac{100000}{8192} = 12.207 \text{ Hz/bin}.$$

Un pic au bin $k = 410$ correspond à :

$$f_{410} = 410 \times 12.207 \approx 5005 \text{ Hz}.$$

Conversion dB. Si $|X[410]| = 2500$, alors :

$$S[410]_{dB} = 20\log_{10}(2500) \approx 67.96 \text{ dB}.$$

Si le frame précédent avait $S_s^{(m-1)}[410] = 64$ dB et $\alpha = 0.25$:

$$S_s^{(m)}[410] = 0.75 \times 64 + 0.25 \times 67.96 = 64.99 \text{ dB}.$$

On voit la stabilisation : le saut brut de +3.96 dB devient +0.99 dB.

7 Détection CFAR et robustification bruit

7.1 CA-CFAR : détail de la construction du seuil

Pour une cellule test k (CUT), on définit :

— G cellules de garde de chaque côté (pour exclure l'énergie du pic),

— R cellules de référence de chaque côté (estimation bruit local).

Le nombre de cellules de référence effectives vaut $N_{ref} \approx 2R$ (hors bords).
Le bruit local estimé en dB :

$$\widehat{N}[k] = \frac{1}{N_{ref}} \left(\sum_{i \in \mathcal{L}_k} S_s[i] + \sum_{i \in \mathcal{R}_k} S_s[i] \right),$$

avec $\mathcal{L}_k, \mathcal{R}_k$ les ensembles de référence gauche/droite.
Le seuil :

$$T[k] = \widehat{N}[k] + \alpha_{CFAR},$$

et la détection binaire :

$$D[k] = \begin{cases} 1, & S_s[k] > T[k] \\ 0, & \text{sinon.} \end{cases}$$

Sens de α_{CFAR}. α_{CFAR} est une marge en dB au-dessus du bruit local. Plus α augmente, plus la probabilité de fausse alarme baisse, mais plus la probabilité de détection des cibles faibles diminue.

7.2 Traitement des perturbations secteur et harmoniques

Le code inclut un masque "mains" : bins proches de hf_{mains} ($h = 1..H$) sont pénalisés/invalidés.
Mathématiquement, pour un bin k, si

$$|f_k - hf_{mains}| \leq W_{mains},$$

alors $D[k] \leftarrow 0$ et $T[k]$ est majoré d'une marge supplémentaire.
Interprétation : on injecte une connaissance *a priori* sur une source d'interférence quasi-déterministe.

7.3 Notch paramétrique

Si activé, un notch logiciel atténue les bins de la bande :

$$\mathcal{B}_{notch} = \left[f_c - \frac{B}{2},\ f_c + \frac{B}{2} \right]$$

avec profondeur D_{notch} (en dB). Concrètement, le code soustrait D_{notch} à $S[k]$ et $S_s[k]$ dans cette bande.

7.4 Plancher de bruit adaptatif : modèle dynamique

Le bruit instantané N_{inst} est la médiane des bins non détectés et non notched. Puis :

$$N_f^{(m+1)} = \begin{cases} N_f^{(m)} + \alpha_{\uparrow}(N_{inst}^{(m)} - N_f^{(m)}), & N_{inst}^{(m)} > N_f^{(m)} \\ N_f^{(m)} + \alpha_{\downarrow}(N_{inst}^{(m)} - N_f^{(m)}), & N_{inst}^{(m)} \leq N_f^{(m)}. \end{cases}$$

Ensuite on force :

$$T[k] \geq N_f + \Delta_{offset}.$$

Lecture "système". $\alpha_\uparrow$ rapide permet de suivre une hausse soudaine du bruit ; $\alpha_\downarrow$ plus lent évite de "décrocher" trop vite après un transitoire. C'est un mécanisme d'hystérésis temporelle sur le plancher.

7.5 Exemple numérique (section 7)

Prenons :

$$G = 4, \quad R = 16, \quad \alpha_{CFAR} = 12 \text{ dB}.$$

Donc $N_{ref} \approx 32$ cellules.

Pour un CUT k, supposons :

$$\widehat{N}[k] = -58.5 \text{ dB}.$$

Le seuil devient :

$$T[k] = -58.5 + 12 = -46.5 \text{ dB}.$$

Si $S_s[k] = -43.0$ dB :

$$S_s[k] - T[k] = 3.5 \text{ dB} > 0 \Rightarrow D[k] = 1.$$

Si un plancher adaptatif vaut $N_f = -50$ dB avec $\Delta_{offset} = 8$ dB :

$$N_f + \Delta_{offset} = -42 \text{ dB}.$$

Le seuil effectif devient alors au moins -42 dB (plus strict que -46.5 dB), ce qui bloque certaines fausses alarmes en bruit montant.

Enfin, exemple de mise à jour adaptative :

$$N_f^{(m)} = -55, \quad N_{inst}^{(m)} = -49, \quad \alpha_\uparrow = 0.2$$

$$N_f^{(m+1)} = -55 + 0.2 \times 6 = -53.8 \text{ dB}.$$

8 Extraction de la cible et estimation cinématique

8.1 Du masque binaire au pic cible

Parmi les bins avec $D[k] = 1$, le code retient :

$$k^\star = \arg \max_{k:D[k]=1} S_s[k].$$

Puis il applique un filtre médian sur l'historique des indices :

$$\tilde{k}^{(m)} = \text{median}\big(k^\star_{m-L+1}, \ldots, k^\star_m\big)\,,$$

($L =$`peak_median_len`).

Ce filtrage est très efficace contre les sauts impulsionnels d'un bin à l'autre.

8.2 Conversion fréquence → vitesse

Une fois $\tilde{k}$ obtenu :

$$f_{peak} = \tilde{k}\frac{F_s}{N_{FFT}}, \qquad v_{raw} = \frac{f_{peak}}{DOPPLER_SCALE}.$$

En km/h :

$$v_{raw,km/h} = 3.6\, v_{raw}.$$

8.3 SNR de la cible

Le SNR affiché est défini (en dB) par :

$$SNR_{peak} = S_s[\tilde{k}] - T[\tilde{k}].$$

Ce n'est pas un SNR "thermodynamique absolu", mais un indicateur de *distance au seuil adaptatif* : excellent pour le monitoring opérationnel.

8.4 Filtre de Kalman : dérivation des équations utilisées

Le vecteur d'état est :

$$\mathbf{x}_m = \begin{bmatrix} v_m \\ a_m \end{bmatrix}, \quad \mathbf{F} = \begin{bmatrix} 1 & dt \\ 0 & 1 \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

Le modèle de mesure est :

$$z_m = \mathbf{H}\mathbf{x}_m + \nu_m, \qquad \nu_m \sim \mathcal{N}(0, R).$$

La prédiction :

$$\hat{\mathbf{x}}_m^- = \mathbf{F}\hat{\mathbf{x}}_{m-1}, \qquad \mathbf{P}_m^- = \mathbf{F}\mathbf{P}_{m-1}\mathbf{F}^T + \mathbf{Q}.$$

Le code implémente $\mathbf{Q}$ du modèle "accélération blanche" :

$$\mathbf{Q} = q \begin{bmatrix} dt^4/4 & dt^3/2 \\ dt^3/2 & dt^2 \end{bmatrix}.$$

La correction :

$$\mathbf{K}_m = \mathbf{P}_m^- \mathbf{H}^T (\mathbf{H}\mathbf{P}_m^- \mathbf{H}^T + R)^{-1},$$
$$\hat{\mathbf{x}}_m = \hat{\mathbf{x}}_m^- + \mathbf{K}_m (z_m - \mathbf{H}\hat{\mathbf{x}}_m^-),$$
$$\mathbf{P}_m = (\mathbf{I} - \mathbf{K}_m \mathbf{H})\mathbf{P}_m^-.$$

Signification des paramètres de réglage.

- q (`kalman_q_accel`) : confiance dans le modèle dynamique (grands changements possibles).
- R (`kalman_r_meas`) : bruit supposé de la mesure Doppler FFT.

Gestion des pertes de détection. Quand aucune mesure n'est disponible :

- pas d'update, seulement prédiction,
- décroissance $v \leftarrow \lambda v$ (`kalman_miss_decay`),
- annulation sous seuil (`speed_floor`),
- reset complet après `kalman_miss_reset_count`.

Ce bloc transforme un estimateur purement probabiliste en estimateur robuste opérationnel, adapté à un capteur réel sujet aux extinctions temporaires.

8.5 Exemple numérique (section 8)

1) Fréquence et vitesse. Avec $N_{FFT} = 8192$, $\Delta f = 12.207$ Hz/bin. Supposons $\tilde{k} = 132$:

$$f_{peak} = 132 \times 12.207 = 1611.3 \text{ Hz}.$$

$$v_{raw} = \frac{1611.3}{161} = 10.01 \text{ m/s} = 36.0 \text{ km/h}.$$

2) SNR au pic. Si $S_s[\tilde{k}] = -37.2$ dB et $T[\tilde{k}] = -49.0$ dB :

$$SNR_{peak} = 11.8 \text{ dB}.$$

3) Pas Kalman (illustratif). Prenons $dt = 0.01024$ s, estimation prédite $v^- = 9.6$ m/s, mesure $z = 10.0$ m/s. Si le gain vitesse $K_0 = 0.35$, alors :

$$v = v^- + K_0(z - v^-) = 9.6 + 0.35(0.4) = 9.74 \text{ m/s}.$$

Le filtre corrige partiellement vers la mesure, sans la suivre brutalement.

9 Publication des résultats et rendu GUI

9.1 Données partagées

Le thread DSP publie (mutex) :

— ADC glissant brut,
— spectre dB, seuil CFAR, masque détections,
— f_{peak}, SNR, vitesse brute et Kalman,
— plancher de bruit adaptatif,
— spectrogramme circulaire,
— FPS, état connexion, pertes de trames.

9.2 Sorties visuelles

Le GUI construit :

— **Oscilloscope** (temps), avec fenêtre temporelle ajustable,
— **FFT + CFAR** (fréquence), avec curseur et marqueur pic,
— **Spectrogramme waterfall** (temps-fréquence),
— **Panneau numérique** (vitesse, SNR, statut cible),
— **Trend Doppler** long terme.

Le spectrogramme est stocké en tampon circulaire 2D, texture OpenGL mise à jour en filtrage `NEAREST` pour une lecture nette des bins.

10 Lecture "professeur" : compromis et limites

10.1 Résolution et latence

La résolution fréquentielle vaut :

$$\Delta f = \frac{F_s}{N_{FFT}}$$

et la résolution vitesse :

$$\Delta v = \frac{\Delta f}{161}$$

Augmenter N_{FFT} améliore Δf, mais augmente coût de calcul et inertie d'estimation. Le hop à 10.24 ms maintient une cadence de rafraîchissement élevée.

10.2 Ce que CFAR résout, et ce qu'il ne résout pas

CFAR traite bien l'hétérogénéité du bruit local, mais :

— il reste sensible à des interférences structurées (secteur, parasites harmoniques),
— il peut dégrader en multi-cibles très proches si les fenêtres de garde/référence sont mal réglées.

D'où l'intérêt des briques complémentaires (notch, adaptive floor, median peak, Kalman).

10.3 Pourquoi vitesse brute + vitesse filtrée

La vitesse brute montre le comportement instantané physique ; la vitesse Kalman fournit une estimation opérationnelle plus stable (pilotage UI, décision humaine).

11 Conclusion

`pc_analyzer` réalise une chaîne DSP radar complète et moderne :

1. acquisition distribuée (STM32 + UDP fragmenté),
2. reconstruction glissante et pré-traitements robustes,
3. analyse spectrale haute résolution,
4. détection adaptative (CFAR + bruit adaptatif + mains/notch),
5. estimation cinématique stabilisée (Kalman),
6. restitution visuelle et métrologique temps réel.

Autrement dit, le logiciel n'est pas une simple FFT affichée : c'est un estimateur radar complet, conçu pour transformer un flux ADC bruité en information vitesse exploitable.